Gmail

Jose Parreño Garcia <joparga3@gmail.com>

[TEST rkw - Paid] Claude Code agent teams: when and how to go multi-agent

2 mensajes

Jose Parreño Garcia from Senior Data Science Lead <joseparreogarcia@substack.com> 2 de mayo de 2026 a las 12:23
Para: joparga3@gmail.com

View in browser

The Senior Data Science Lead newsletter

Hi! It's Jose from Senior Data Science Lead newsletter, where I write about senior data science leadership, sharing hard-earned lessons from building teams, systems, and real-world machine learning.

Because you are a paid subscriber, you are directly supporting this work — and you get full access to the practical tools built alongside the writing:

- ***The full Data Storytelling course**, focused on turning analysis into insights that land with executives. Because today, anyone can build a chart with AI, but very few people can build the right chart.*

📊 Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

Read full story

- *50+ Notion templates for hiring, onboarding, mentoring, and planning built for Data and Tech Leads. As a paid subscriber, you get **50% off** with code **PAIDSUBS-03PL**.*

💼 50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

Read full story

And as always, thank you for supporting the newsletter — your support is what keeps it independent, practical, and focused on real-world impact.

Claude Code agent teams: when and how to go multi-agent

4 patterns, 8 failure modes, and a decision framework from real multi-agent usage.

JOSE PARREÑO GARCIA

MAY 2 · PAID · DRAFT

READ IN APP

ADD THE AGENTS BLOG POST INDICATED IN THE INTRO!!

This week I finished building an orchestrated team of agents interacting with each other within the claude code ecosystem. I actually liked how the final version of the system works, but it is not as simple as it sounds to get one working.

And I fear many people are jumping into building agent teams without fully understanding the trade-offs. I get it, agent teams sound like the natural next step once you have subagents working.

Let me be clear: **they are not**.

Agent teams are a coordination layer with a real token cost, known failure modes, and a narrow set of problems they solve well.

The most useful conclusion from some months of real multi-agent usage is not very glamorous: most systems should start with 1 well-configured agent, good tools, and disciplined context management. That's it. Add more agents *only* when you can name a specific reason to do so. Anthropic's own guidance on building effective agents says as much directly: *"We recommend finding the simplest solution possible, and only increasing complexity when needed."*

This post is about how to think about that decision clearly, and what to do once you have made it.

If you have not yet read the post on Claude Code subagents, start there.

ADD IT HERE!!

This post assumes you know what a subagent is, how it differs from a skill, and why context isolation is the primary reason to use one. The question here is different: when does one subagent reporting back to you stop being enough and what do you build instead?

What will we cover in this post?

- **Why most teams don't need multiple agents yet.** The common failure pattern: reaching for agent teams because they feel more powerful, not because there is a specific problem they solve.
- **When 1 agent stops being enough.** 4 concrete triggers, and why each one matters.
- **The 4 patterns for combining agents.** A provider-agnostic taxonomy: orchestrator, pipeline, parallel specialists, swarm.
- **The Anthropic stack for agent teams.** A clear picture of what exists today, what is stable, and what is still experimental or preview-only.
- **Claude Code agent teams in practice.** What the feature actually is, how it works, and how it differs from subagents.

- **How agents communicate.** 3 forms of communication, 4 timing patterns, and *what* to actually pass between agents.
- **Why agent teams fail.** 8 failure modes with practical mitigations (the section most posts on this topic skip).
- **Native vs DIY vs hybrid.** A decision framework for choosing how to orchestrate, without the vendor framing.

Let's get started!

Why most teams don't need multiple agents yet.

Most people who reach for agent teams are solving the wrong problem.

The reasoning usually goes: subagents worked well, the system is getting more complex, agent teams are the next step. But that logic conflates capability with complexity. An agent team does not give you a more capable system, it gives you a more *distributed* one. Distribution is a cost, not an upgrade.

Anthropic's building effective agents post makes the economics explicit: workflows that are not agentic are more predictable, cheaper, and easier to debug. The guidance is direct — *"if you can write a function to do the job, do that instead of calling in a committee of probability distributions."* I really like this statement because people who dont understand how LLMs work miss the point that they are probabilistic, which means, that even if you ask for an explicit action, the LLM might or might comply with it correctly. Multi-agent

coordination adds overhead and compounds error risk. Each agent you add is another component that can fail, hallucinate, or produce output that the next agent misinterprets. So basically, you are literally deciding to compound the probabilities of error at scale.

The single-agent ceiling is higher than most people think. A well-designed agent with the right tools, a focused system prompt, clean subagent delegation for heavy work, and proper context management will handle a lot. The ceiling is reached when one of 4 specific things is true:

1. The work is too large to fit in one context window, even with subagent delegation.
2. Different parts of the work require genuinely different specialisations — not just different prompts, but different tool access and behavioural constraints.
3. Parts of the work can run in parallel, and wall-clock time matters.
4. The quality of the output requires independent critique (for example with a second agent that has not seen the first agent's reasoning).

None of those conditions is about capability. They are about *architecture*. When one of them is true, adding agents is justified. When none of them is true, you are adding friction.

If you cannot point to one specifically, the single-agent path is still open.

When 1 agent stops being enough.

The 4 conditions named earlier deserve concrete examples.

- **Context isolation.** The work requires reading dozens of files, running extensive searches, and holding large intermediate outputs — and none of that intermediate work needs to persist in your main session. A subagent handles this. But if the investigation itself is so large that even a subagent's context fills, or if the result needs to be broken into independent parallel investigations, you are looking at multiple agents. This is the most common legitimate trigger.
- **Specialisation.** The work requires not just different instructions, but different tool access and different behavioural constraints. A code reviewer that cannot write files or run shell commands is a categorically different risk profile from a code generator. You can still encode these differences as separate subagents, each with a tailored system prompt and a restricted tool list, so careful with taking specialisation as the only dimension to decide when to upgrade to agent teams. In fact, the Claude Code agent teams documentation is explicit that subagents excel here — isolated contexts, summaries back to the caller, task-specific instructions.
- **Parallel work.** The task decomposes into independent subtasks that do not share mutable state. 3 agents investigating different angles of a codebase bug simultaneously. 4 agents reviewing different chapters of a document. This is where multi-agent systems produce genuine wall-clock time savings because they run at the same time. The cost though is more total tokens. The trade-off is legitimate when wall-clock time matters more than token spend.

- **Independent critique.** The quality of the output requires a second opinion that has not been contaminated by the first opinion's reasoning. An evaluator agent that reads only the final output, not the reasoning that produced it. A red-team agent that actively tries to find flaws rather than validate claims. This is the pattern behind Anthropic's multi-agent research system, where parallel subagents investigate different aspects and a lead agent synthesises independently.

The moment one of these conditions is clearly true — and you can state which one — is when adding agents earns its cost.

The 4 patterns for combining agents.

Now, say that you do want to move ahead with multi-agent systems. If you go and research where to start you will find yourself in a bit of a problem, as multiple frontier labs have been developing multi-agent patterns and solutions differently. Still, there are some common denominators worth exploring.

What follows is a synthesised taxonomy based on the patterns that appear across Anthropic's workflow documentation and Anthropic Engineering's production experience. Just to be clear, what I cover below is not Anthropic's official taxonomy, it is an analytic synthesis that maps back to what the documentation describes.

Sequential pipeline.

Agents or stages run in a defined order. Later stages consume earlier outputs. This is the easiest pattern to explain to a technical audience because

it looks most like ordinary software. Research then synthesis. Drafting then critique. Extraction then validation.

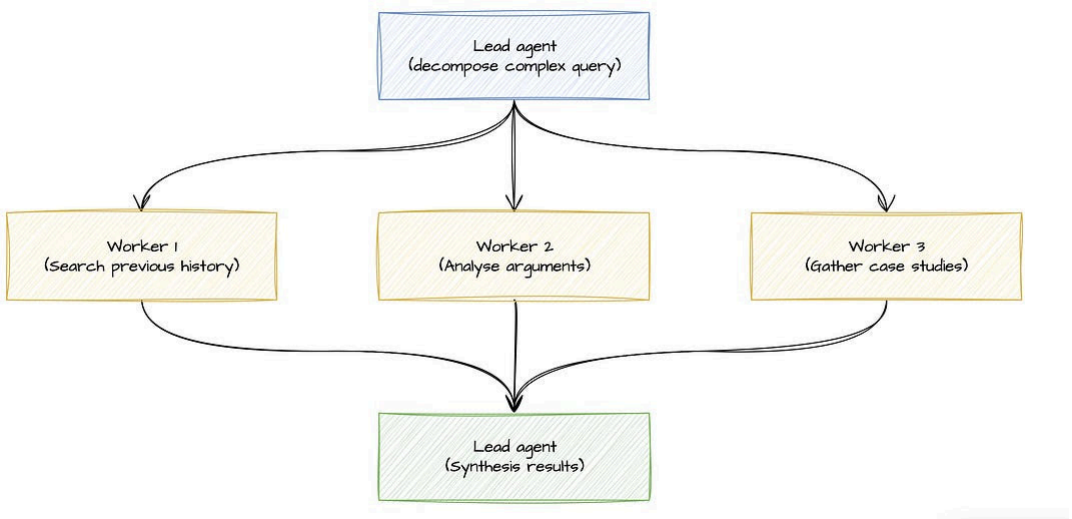
The strength is that contracts between stages are explicit. The weakness is that mistakes cascade: if the first stage extracts the wrong thing, later stages can be wrong at great expense. Validation gates between stages are not optional on this pattern — they are what keeps the pipeline from running confidently in the wrong direction. The best examples of validation gates are json contract schemas and other artifacts to ensure lineage in the execution process.

Central orchestrator or planner-executor.

1 lead agent decomposes the task, delegates to worker agents, and synthesises the results. Workers do not coordinate with each other — they report back to the orchestrator. This is the most common and most useful default pattern. Use it when the subtask structure is not known in advance but clear top-level ownership matters. Anthropic's Research feature uses exactly this pattern: a lead agent analyses the query, spawns specialised subagents in parallel, and later synthesises findings.

Avoid it when the workflow is already fixed enough to script. If you know the steps in advance, a pipeline is simpler and more predictable.

The main failure mode is **orchestrator failure**: a lead agent with vague delegation prompts causes workers to duplicate each other's searches, miss important boundaries, or produce outputs that do not compose. The fix is explicit: give each worker an objective, an expected output format, source guidance, and task boundaries.

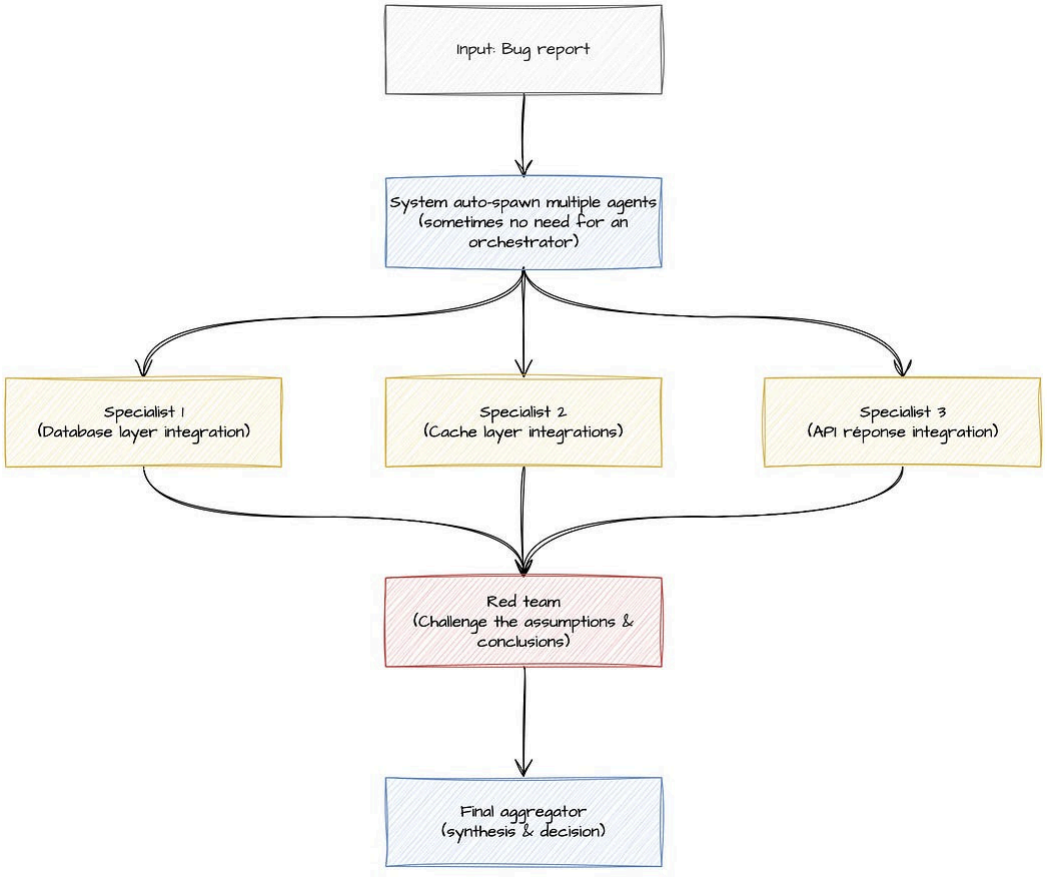


Parallel specialists, debate, and red team.

Multiple agents examine the same task or related subtasks simultaneously, then vote, critique, or aggregate. Use this when robustness matters more than tidiness: competing hypotheses about a bug, adversarial review of a policy decision, security review from multiple angles.

The cost is obviously that more agents means significantly more tokens, more aggregation work, and a risk of false confidence from consensus. Anthropic's research system had to add effort budgets and clearer delegation

to stop workers fanning out without discipline. This pattern only makes sense when the domain genuinely requires it.

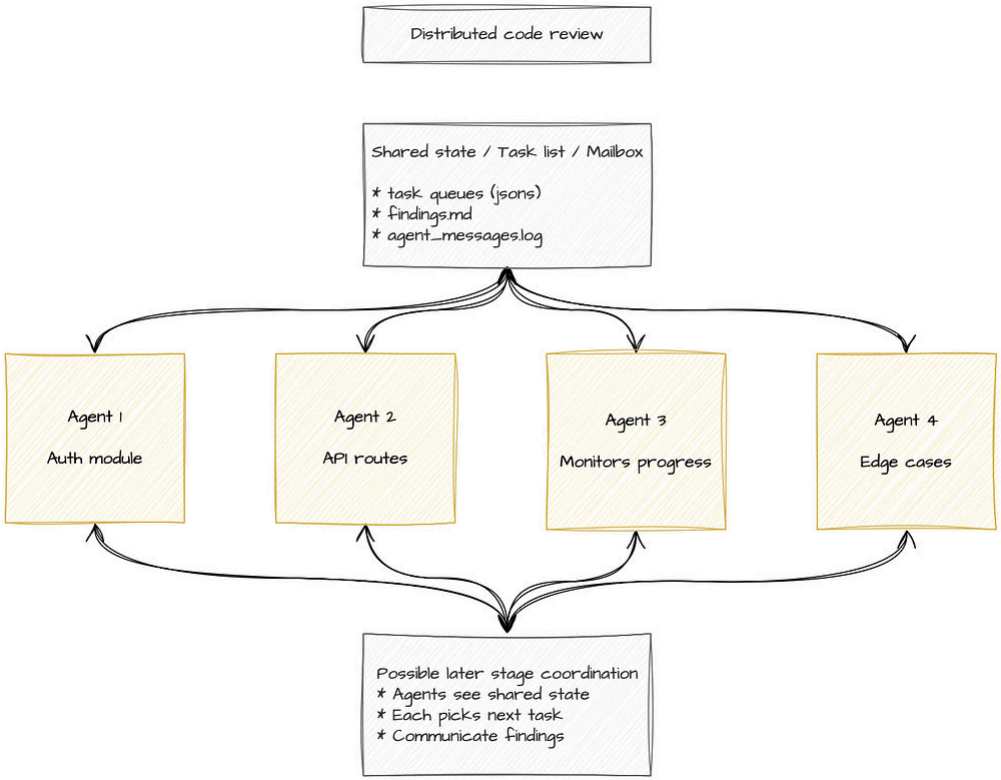


Swarm or decentralised shared-context systems.

Coordination emerges through shared state, shared message threads, or environment changes rather than one strict leader. This is where multi-agent systems start to become distributed systems, with the associated complexity: ownership ambiguity, shared-state race conditions, debugging difficulty. Use

this when the problem is genuinely open-ended and agents must interoperate across session or runtime boundaries. Avoid it when you need easy accountability or tight compliance.

For most practitioners, the first 2 patterns cover almost everything. Below you can find the summary of the 4 patterns.



A summary of the 4 agent team patterns

Created with Datawrapper

Attribute	Sequential Pipeline	Central Orchestrator	Parallel Specialists	Swarm
What Is It	Agents or stages run in a defined order. Later stages consume earlier outputs.	One lead agent decomposes the task, delegates to worker agents, and synthesises results. Workers report back to the orchestrator.	Multiple agents examine the same task or related subtasks simultaneously, then vote, critique, or aggregate.	Coordination emerges through shared state, shared message threads, or environment changes rather than one strict leader.
When to Use	When the workflow is known and linear. Research then synthesis. Drafting then critique. Extraction then validation.	When the subtask structure is not known in advance but clear top-level ownership matters. When you need ad hoc decomposition.	When robustness matters more than tidiness. Competing hypotheses about a bug. Adversarial review of policy decisions. Security review from multiple angles.	When the problem is genuinely open-ended and agents must interoperate loosely. When you need emergent coordination without a single point of control.
When to Avoid	When stages require parallel execution or concurrent work. When you need agents to self-coordinate mid-task.	When the workflow is already fixed enough to script. If you know the steps in advance, a pipeline is simpler and more predictable.	When token cost is constrained or wall-clock time is not a priority. When agents fanning out would be wasteful.	When you need easy accountability or tight compliance. When debugging matters more than flexibility. When ownership needs to be clear.
	Cascading failure: mistakes in early stages	Orchestrator failure: vague		Distributed system

Common Failure Mode	compound down the pipeline. A wrong extraction makes everything downstream beautifully wrong. Validation gates between stages are not optional.	delegation causes workers to duplicate searches, miss boundaries, or produce outputs that don't compose. Fix: explicit objective, output format, source guidance, and task boundaries for each worker.	False confidence from consensus: more agents = more tokens, more aggregation work, real risk that agreement masks a shared misinterpretation. Requires discipline and effort budgets.	complexity: ownership ambiguity, shared-state race conditions, debugging difficulty. Most practitioners should avoid unless the problem genuinely demands it.

Created with Datawrapper

The Anthropic stack for agent teams.

Now, before you go on and start building your own stack, let’s review if Anthropic has something that can be used (and if we can learn from their ideas). Anthropic now offers multiple surfaces for agentic work, and they are not interchangeable.

The clearest way to think about it is as a ladder — from raw control to hosted management:

1. **Messages API** — you own the loop entirely. Tool use exists but you implement the execution logic yourself. No multi-agent support is built in. This is the most flexible tier and the most work.
2. **Claude Agent SDK** — Anthropic’s agent loop, tools, context management, subagents, hooks, and cost tracking run in infrastructure you operate. Subagents are supported natively. Multi-agent patterns

beyond subagents are still largely your design. Crucially, the Agent SDK authenticates through Bedrock, Vertex AI, and Azure Foundry, making it the most portable tier.

3. **Claude Managed Agents** — Anthropic hosts the runtime. You get managed infrastructure, containers, prompt caching, compaction, stateful sessions, event history, and built-in tools. Single-agent usage is mature. Multi-agent sessions are a separate, gated preview.
4. **Managed Agents multiagent** — a Research Preview, access-gated, with a significant constraint: the current design supports only one coordinator level. A coordinator can call other agents, but called agents cannot themselves call further agents. That is multi-agent orchestration, but it is not yet a general-purpose recursive agent society.
5. **Claude Code subagents** — isolated worker contexts that run inside a single main session and report results back. An official feature, not peer-to-peer teams.
6. **Claude Code agent teams** — as of April 2026, this feature is experimental and disabled by default. It requires a specific environment variable to enable. Known limitations include session resumption behaviour, task coordination edge cases, and the possibility that resumed sessions refer to teammates that no longer exist. This is the honest picture.

What this means practically:

- *“Claude supports agent teams”* is true.

- *“Claude has production-grade, general-purpose native multi-agent orchestration across all surfaces”* is not accurate.

Claude Code agent teams in practice.

Lets start with a short refresher of subagent vs agent teams:

- **Subagents** run inside a single main session. Your main Claude Code instance spawns them, they do work in isolation inside their own context window, and they return results. You orchestrate. The main session never absorbs the work they did to produce the result.
- **Agent teams** are separate Claude Code sessions that communicate with each other. Each teammate has its own context window, its own session state, and access to a shared task list and a mailbox for direct teammate messaging. Teammates can assign tasks to one another and self-coordinate without waiting for the lead to pass each instruction. The lead sets direction; teammates can act on it and talk to each other.

So, how to create these agent teams?

The practical setup

1. A team configuration lives at `~/.claude/teams/{name}/config.json`.
2. A shared task list lives at `~/.claude/tasks/{name}/`.

3. Teammate definitions are reusable agent files — essentially subagent definitions that get assigned teammate roles.
4. Hooks can serve as quality gates between task transitions.

2 architectural notes that matter for design:

First, subagents can maintain their own auto memory. This is useful for specialisation, but the moment you split work across multiple agents with their own memory, you are making an architecture decision: which knowledge stays local, which is shared, which is summarised back. You are facing **a system design question.**

Second, the documentation is direct about when agent teams are worse than subagents: sequential tasks, same-file edits, work with many dependencies between steps. If agents need to hand off the same artefact sequentially rather than work in parallel, subagents with a sequential pipeline are simpler and more predictable.

As of April 2026, agent teams are experimental and disabled by default. Enable them only when you have a clear reason from the list above, and expect to encounter edge cases that have not yet been smoothed over.

How agents communicate.

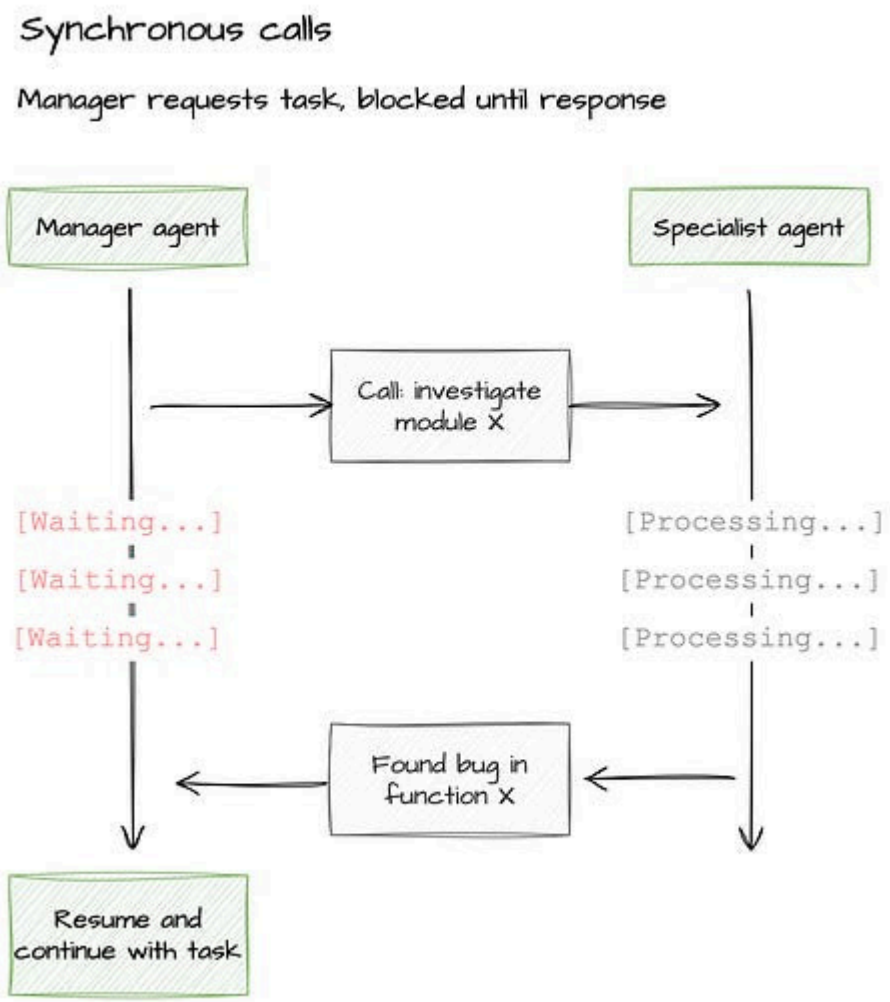
Agent communication is often treated as a prompting problem. At scale, it is a systems problem.

There are 3 broad forms of communication between agents.

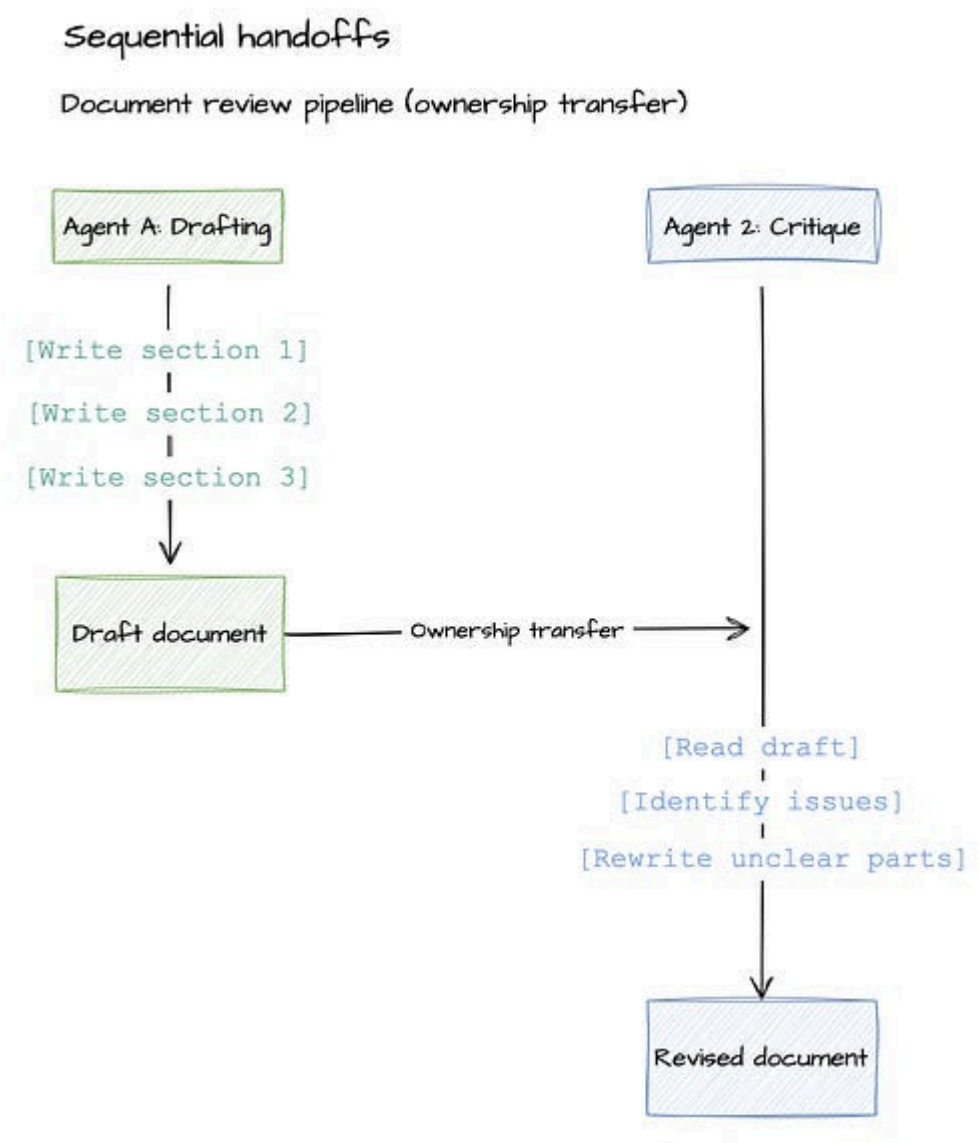
1. **Plain language messages** — the most obvious and the least reliable at scale. Claude Code agent teams let teammates message each other directly through the “mailbox”. This is flexible, but natural language is less precise than structured invocation, which means more room for misinterpretation between agents.
2. **Structured invocation** — JSON Schema tool definitions and strict tool use. Anthropic’s tool definitions guarantee schema-conformant arguments. This is the layer that makes agent-to-tool communication reliable, and it is the same mechanism that makes agent-to-agent calls more precise when modelled as tool calls rather than plain messages.
3. **Shared state or artefacts** — the most underrated form of communication. Anthropic’s multi-agent research system explicitly recommends having subagents write outputs to the filesystem or an external system and return lightweight references rather than piping everything through the lead agent’s context. The reason is straightforward: a file is auditable, versioned, and can be inspected directly. A message summarising a file is none of those things.

The timing of communication matters as much as the format. 4 patterns:

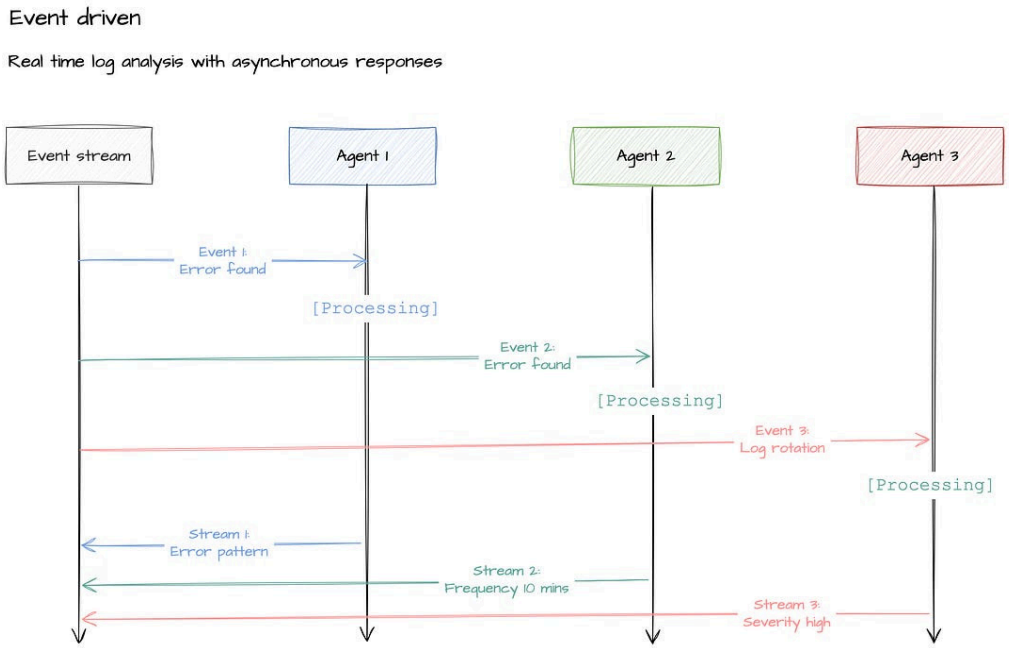
1. **Synchronous calls** — a manager calls a specialist, waits, then continues. Simple to reason about, but could create bottlenecks. Anthropic’s research team found that synchronous lead agent execution is blocked by slow workers and prevented mid-flight steering.



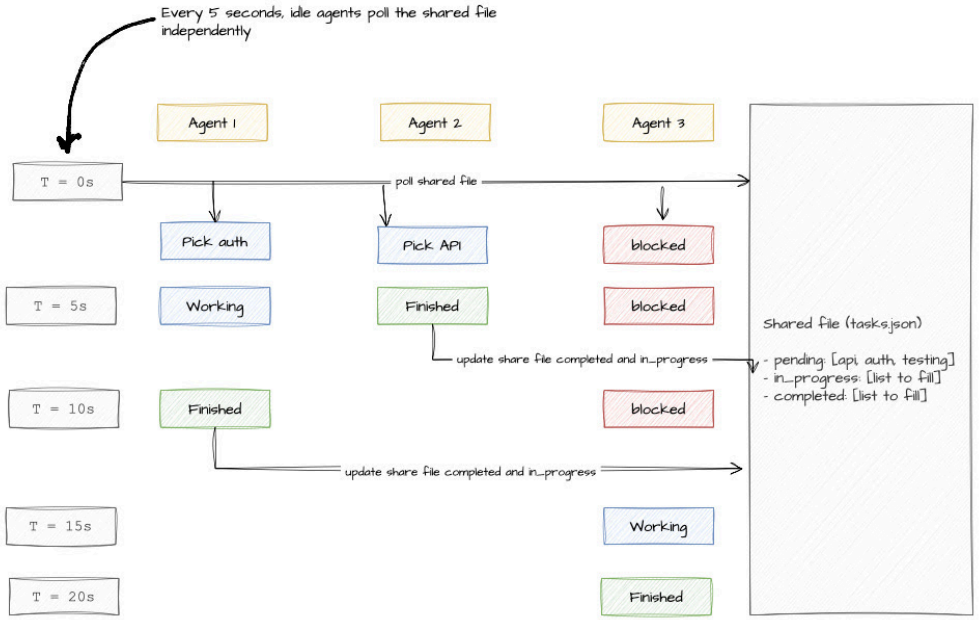
2. **Sequential handoffs** — ownership transfers from one agent to another. One agent produces a result; the next picks it up. OpenAI formalises the distinction between handoffs (where the specialist owns the next response) and agents-as-tools (where the manager stays in control). The distinction matters for accountability.



3. **Event-driven or streaming** — agents respond to events rather than synchronous calls. Better for long-running work, harder to debug and test. Sometimes hooks can help here.



4. **Shared-state polling** — the least glamorous and often the most robust. Agents check a shared file or data store for progress, write updates to it, and proceed. Anthropic's long-running harness work uses JSON feature lists, progress files, and git commits as durable shared state. Human-in-the-loop checkpoints fit naturally here.



As you can see, what agents pass to each other shapes the entire system's reliability. On one hand, passing the full conversation history maximises continuity but grows cost and latency linearly. On the other, a compressed summary is often better — it is what Claude Code subagents do by design. But probably, passing **task-specific artefacts** — structured output files, code patches, feature lists in JSON, stored reports — is usually the practical sweet spot. The receiving agent can inspect the artefact directly rather than trusting a summary, which makes the system auditable in a way that conversational handoffs are not.

This means that the what is passed also shapes the how and when it is passed. Therefore, reinforcing the idea that when you are building agent teams, you are facing a system design problem. And those are not as easy to solve as a linear orchestrated flow.

Why agent teams fail.

This is the section most multi-agent tutorials skip. One source that I found use Anthropic's engineering notes. They read less like a research paper and more like a diary of a team that discovered, one bug at a time, that extra agents multiply management problems. Here are 8 things I extract from them:

- **Coordination drift.** Vague delegation causes workers to duplicate each other's searches, miss important boundaries, or produce outputs that do not compose. Anthropic's research system post found this directly: the solution was to teach the orchestrator to specify objective, output format, source guidance, and scope boundaries for each worker. If you do not define ownership, agents will improvise it — and they improvise it badly (guess what, happens the same with humans).
- **Hallucinated ownership.** One agent assumes another has checked something when nobody has. The fix is explicit completion artefacts: task hooks, end-to-end verification before marking anything done, and durable state files that record what has actually been confirmed rather than what was assumed.
- **Silent intermediate failure.** Debugging agents required full production tracing because there was no way to tell whether a failure came from bad search queries, poor source quality, or a tool issue. The top-level session stream is only a condensed view. To understand what a called agent actually did, you have to inspect the worker-level history. Instrument traces from the start, not after the first production incident.

- **Context bloat and memory pollution.** The context window is a finite resource. Shared memory that grows stale or noisy degrades the entire system. Anthropic's context engineering work frames the problem precisely: every tool call, every file read, every partial analysis lands in context and stays there. Across a team of agents, this compounds. The mitigations are well-established: summarise completed phases, isolate heavy work in subagents, prefer small focused memory files over large shared ones.
- **Schema mismatch and brittle prompt contracts.** Agents exchanging data via natural language are fragile. The second agent misinterprets a summary from the first; the error propagates silently. Anthropic's strict tool use exists precisely because non-strict tool calls can produce incompatible types or missing fields. Narrow agent/tool contracts reduce the surface area where this can happen (who would have thought that basic engineering contracts would be the solution for AGI…)
- **Tool misuse and security failure.** More open environments create more entry points for prompt injection. More tools create more ways for an attacker to act once inside. Anthropic's trustworthy agents research identifies prompt injection as a named cyberattack vector, not an edge case. METR's red-teaming of Anthropic's internal agent monitoring systems found novel security vulnerabilities even in a well-resourced internal system. The mitigations: limit permissions, keep tools scoped to what the task actually requires, preserve human approval points for consequential actions.
- **Cost explosion and latency stacking.** Agent teams consume significantly more tokens than a single session. Parallelism can reduce

wall-clock time, but it does not reduce total token spend — it increases it. Running multiple agents against a task is faster by spending more structured effort in parallel, not by spending less. If the task does not justify that cost, a single well-designed agent is faster and cheaper.

- **Evaluation difficulty.** Many different internal trajectories can end in the same successful-looking final output. If your only test is whether the final prose sounds plausible, you are not evaluating the system — you are admiring it. Anthropic's recommendation: evaluate whether the correct final state was achieved, not whether the intended process was followed. The process will vary; the outcome is what you can actually check.

Native vs DIY vs hybrid.

Now, the golden question. If I did want to build agent teams, should I do them myself or use an out-of-the-shelf solution?

The choice between building your own orchestration and using Anthropic's native features is an architecture trade-off between 3 options (I am sure there are more, but I am being opinionated here):

1. **Native managed** means Anthropic owns the runtime and coordination surface — Claude Managed Agents and, in narrower ways, Managed Agents multiagent or Claude Code agent teams. The value is lower engineering overhead, a hosted harness, and infrastructure you do not have to maintain. The cost is constrained portability (Managed Agents is available only through the direct Claude API), and the multi-agent

surface is either experimental (agent teams) or gated preview (Managed Agents multiagent). And of course, dependency on a third party. The right choice when infrastructure is the pain point and Anthropic-specific constraints are acceptable.

2. **DIY** means you own the orchestration logic and usually the runtime too — the Messages API is the purest example. Highest control, highest engineering cost. The right choice when your business logic is unusual, compliance is strict, routing is domain-specific, or you need genuine provider portability.
3. **Hybrid** means Anthropic provides the loop and tools, but you architect the system — the Claude Agent SDK is the clearest example. You get Anthropic's agent loop, context management, hooks, subagents, and observability. You still own the higher-level design. The Agent SDK authenticates through Bedrock, Vertex AI, and Azure Foundry, which is a meaningful portability advantage over native managed. The right choice when you want Anthropic's loop but do not want to marry your entire architecture to Anthropic's current opinion of how multi-agent systems should work.

The practical summary: use native managed when infrastructure is the pain; use DIY when topology and policy are the pain; use hybrid when you want Anthropic's primitives but need to keep room for your own architecture decisions.

Closing thoughts

The frame most people bring to agent teams is capability: more agents means more capability. The frame that actually helps is ***architecture***: *what problem does adding an agent solve, and what does it cost?*

Subagents define who does the work, in what context, and with what tools. Agent teams add a coordination layer on top of that — shared task state, direct teammate messaging, parallel execution with self-coordination. That layer is powerful *only* when the problem specifically requires it.

One well-configured agent team, set up for a real use case, is a reasonable week's work. The complexity can grow from there, but it does not have to start there. The research, the failure mode catalogue, and the communication patterns in this post are reference material — not a prescription to implement everything at once.

Start by naming which of the 4 triggers is actually true. If none of them is, the single-agent path is still the right one.

Now, I want to hear from you

Agent teams sit at an interesting intersection of system design and prompt engineering — and the right answer often depends on constraints that are specific to your situation.

- Have you hit a genuine wall with single-agent + subagents that pushed you toward agent teams? What was the specific trigger?

- Which failure mode from the list above have you run into in practice? Coordination drift and context bloat are the ones I hear about most often.
- For those of you running Claude Code agent teams in production: what has surprised you most about how the feature behaves versus how you expected it to behave?

I'd love to hear more from how are you using agent teams applied to your professional production systems.

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA · OCTOBER 15, 2024

Read full story

References

[1] Building effective agents — Anthropic's canonical guide to agentic system design; primary source for the anti-hype framing, workflow patterns, and the recommendation to start with the simplest solution.

[2] Claude Code: Orchestrate teams of Claude Code sessions — Primary official reference for the Claude Code agent teams feature; covers use cases, architecture, token overhead, and when subagents are the better choice.